

Azure Data Lakehouse Implementation: Medallion Architecture Engineering

Executive Summary By Marra Mohamed

The transition to a Data Lakehouse architecture on Microsoft Azure represents a strategic effort to process massive data volumes with high velocity and reliability. This project implements a modern data stack designed to ingest heterogeneous data from relational databases (Azure SQL) and REST APIs, centralizing them in a secure storage environment.

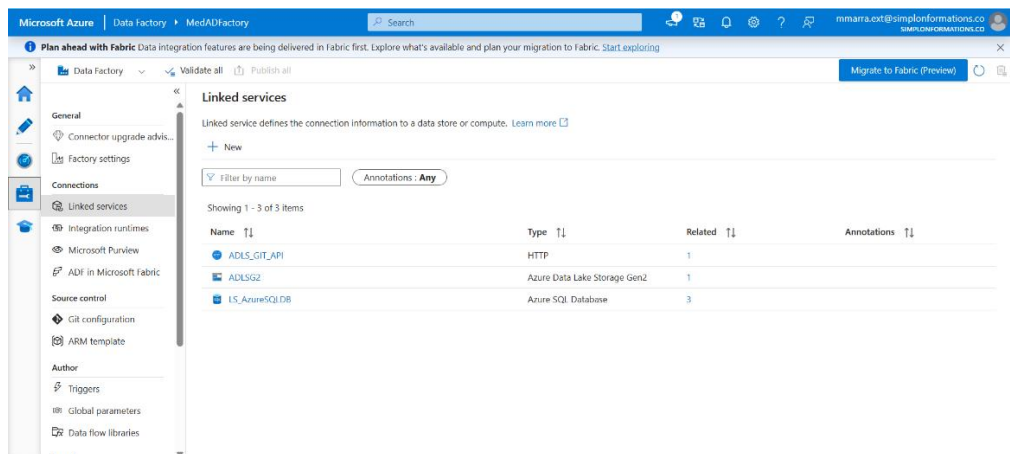
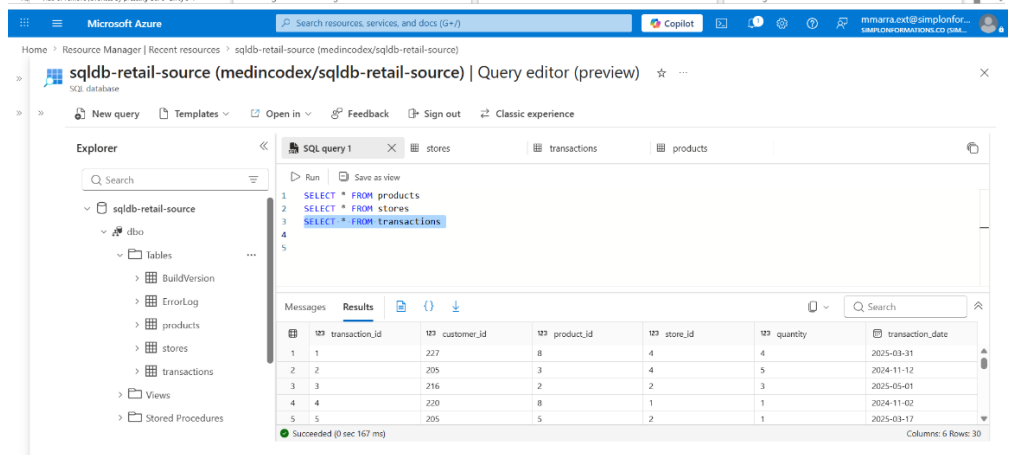
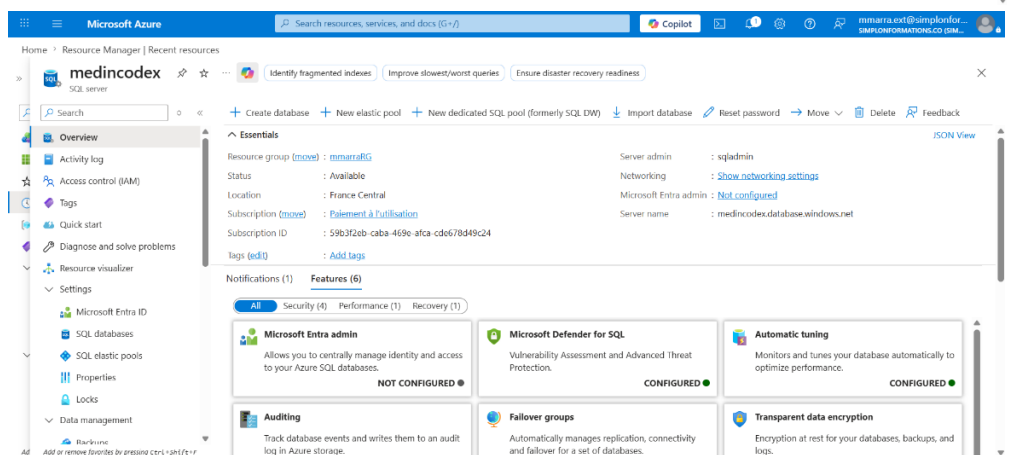
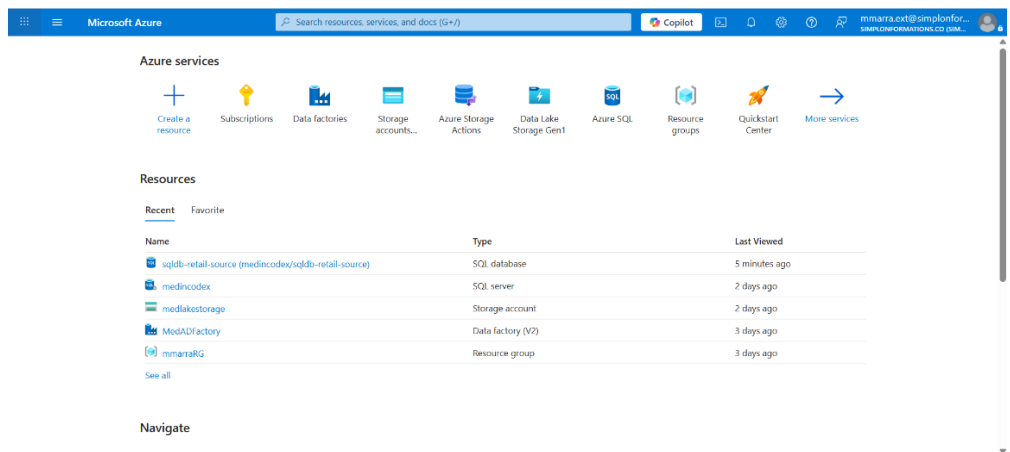
The core of the strategy is the **Medallion Architecture** (comprising Bronze, Silver, and Gold layers), which ensures total data traceability and high quality. Key outcomes include the establishment of a "Single Source of Truth" in the Silver layer and the creation of high-performance, aggregated "One Big Table" (OBT) models in the Gold layer for Business Intelligence (BI) consumption. The architecture achieves scalability by decoupling storage (Azure Data Lake Storage Gen2) from compute (Databricks), reducing "Time-to-Insight" through parallelized orchestration in Azure Data Factory.

Technical Infrastructure and Tooling

The architecture is built exclusively on Azure Cloud-native services, selected for their interoperability and security.

Core Components

Component	Role	Specific Usage
Azure SQL Database	Source (OLTP)	Simulates transactional systems (ERP/CRM) containing Products, Stores, and Transactions.
External REST API	Source (JSON)	Provides semi-structured Customer data for ingestion.
ADLS Gen2	Storage Foundation	Hierarchical storage optimized for Big Data analytics, secured via Access Keys.
Azure Data Factory (ADF)	Orchestrator	Manages ETL/ELT schedules, triggers, and pipeline monitoring.
Databricks (Apache Spark)	Compute Engine	Executes PySpark scripts for distributed data transformation and Delta Lake management.



Phase I: Raw Data Ingestion (Bronze Layer)

The screenshot shows the Microsoft Azure Data Factory interface for a workspace named 'MedADFactory'. The top navigation bar includes the Microsoft Azure logo, the workspace name, a search bar, and user information. The main area features a 'New' button and four primary data engineering tasks: Ingest (Copy data at scale once or on a schedule), Orchestrate (Code-free data pipelines), Transform data (Transform your data using data flows), and Configure SSIS (Manage & run your SSIS packages in the cloud). Below these, a 'Recent resources' table lists the following:

Name	Type	Last opened by you
PL_Ingest_To_Bronze	Pipeline	a minute ago
DS_SQL_Transactions	Dataset	3 minutes ago
DS_SQL_Stores	Dataset	3 minutes ago
DS_SQL_Products	Dataset	3 minutes ago
DS_HTTP_Customers	Dataset	3 minutes ago
DS_ADLS_Parquet	Dataset	3 minutes ago

This screenshot displays the 'PL_Ingest_To_Bronze' pipeline in the editor. The left sidebar shows 'Factory Resources' with a tree view of Pipelines (1) and Datasets (5). The 'Activities' pane on the right lists various data integration options. The main canvas shows a 'Copy data' activity with three input datasets: 'Copy_Products', 'Copy_Stores', and 'Copy_Customers', all of which are mapped to a single output dataset, 'Copy_Transactions'. The bottom of the editor includes tabs for Parameters, Variables, Settings, and Output.

The screenshot shows the 'medlakestorage' Storage account page in the Azure Resource Manager. The left sidebar contains a 'Resource Manager' tree with a search bar and a list of resources including 'medlakestorage'. The main area displays the 'Containers' section, showing a list of containers. The table below lists the containers:

Name	Last modified	Anonymous access level	Lease state	Storage connector
\$logs	02/06/2026 14:43:25	Private	Available	-
medlakestorageee	02/06/2026 16:18:18	Private	Available	-

The Bronze layer's primary objective is to capture the complete history of source data in its raw state without any business-logic alterations. This allows for data reprocessing or "replaying" transformations if required.

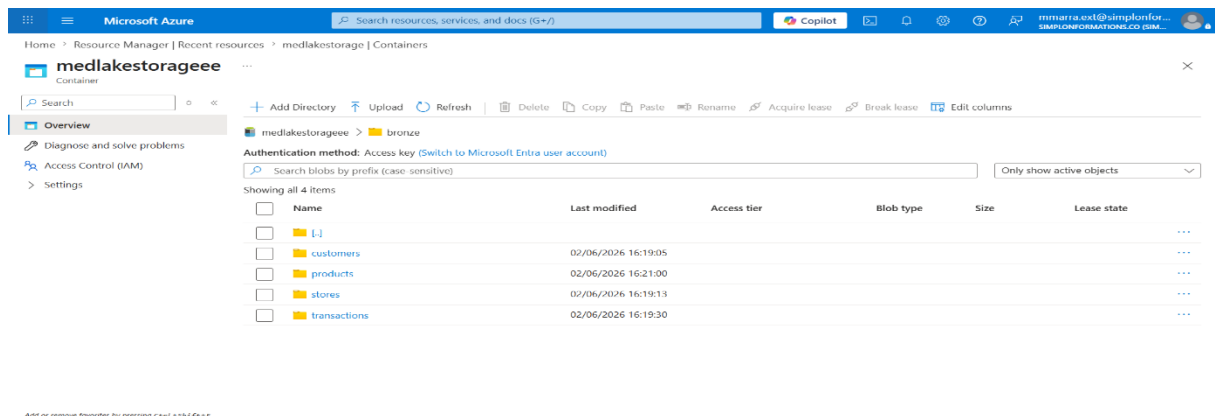
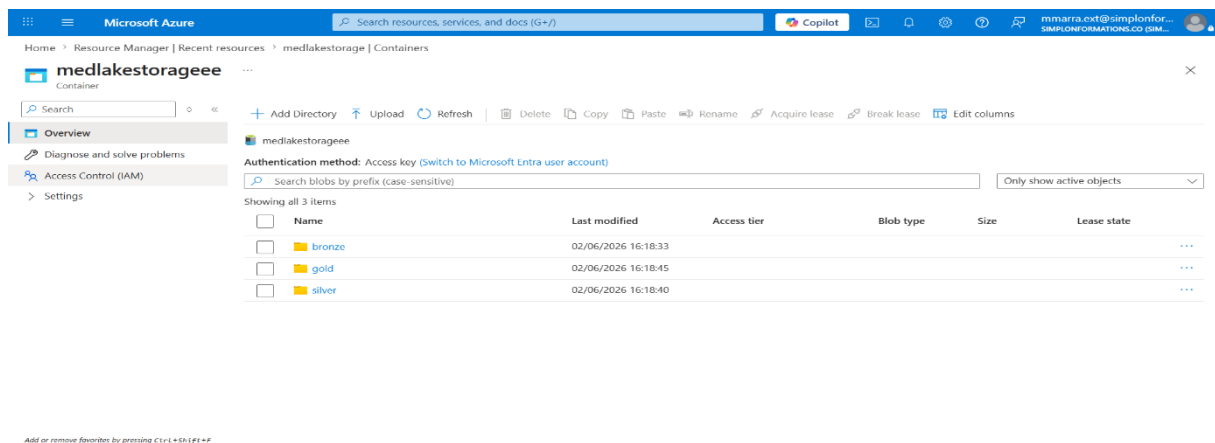
Connection Security

Access between Azure Data Factory and various sources is managed through **Linked Services**. This centralized approach prevents the exposure of passwords or keys within the pipeline code, enhancing overall security governance.

Orchestration Strategy: PL_Ingest_To_Bronze

The ingestion pipeline utilizes a hybrid execution strategy to maximize Data Integration Units (DIU) and reduce latency:

- **Parallel Execution for Dimensions:** Ingestion activities for Copy_Products, Copy_Stores, and Copy_Customers are triggered simultaneously.
- **Dependency Constraints for Facts:** The Copy_Transactions activity is subject to a strict success condition. It only executes after the three dimension activities are successfully completed. This ensures referential integrity, preventing the ingestion of millions of transactions before the necessary reference data is available.



Phase II: Cleansing and Normalization (Silver Layer)

Once raw data is stored in ADLS Gen2, Databricks is employed to transform the Bronze data into a "Single Source of Truth."

Data Quality Operations

PySpark scripts are utilized to apply rigorous quality rules:

- **Deduplication:** The `.dropDuplicates()` method is used to eliminate any redundancy introduced during ingestion.
- **Null Value Management:** Records missing critical identifiers (e.g., a transaction without a Customer ID) are removed using `.dropna()` to ensure healthy joins.
- **Type Casting:** Fields are strictly converted, such as transforming string-based dates into formal `DateType` formats.

Governance and Schema Enforcement

The data is saved in Parquet/Delta format. A critical feature implemented is **Schema Enforcement** via the `overwriteSchema` option. This validates and dynamically updates table structures, preventing data corruption from unanticipated schema changes in the source systems.

Phase III: Business Intelligence and Aggregation (Gold Layer)

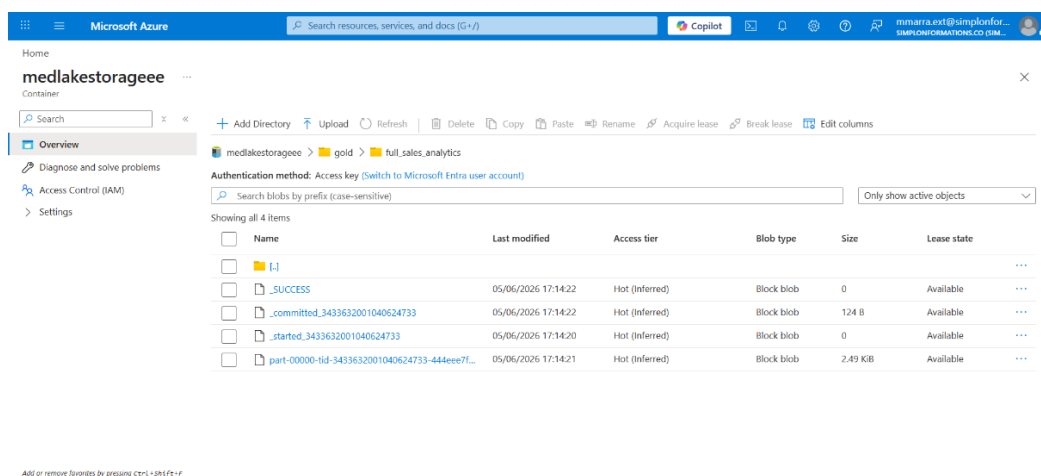
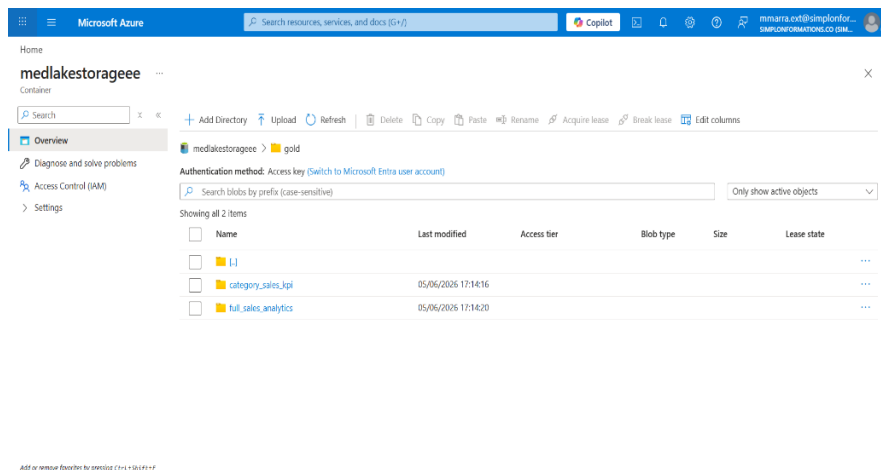
The Gold layer represents the final stage where technical engineering meets business requirements. Data is refined into specific analytical use cases.

KPI Construction

Using PySpark joins (`.join(..., "left")`), transaction data is enriched with product metadata. New metrics are calculated "on the fly," such as **Revenue** ($\text{Quantity} \times \text{Price}$). Aggregation functions (`groupBy, sum`) are then applied to generate category-specific sales reports.

OBT (One Big Table) Modeling

To optimize performance for BI tools like Power BI, the architecture generates "denormalized" flat tables. By pre-calculating complex joins across millions of rows within Databricks' distributed environment, the system provides a fluid user experience and avoids heavy compute loads during dashboard refreshes.



Conclusions and Evolution Roadmap

The implemented Medallion Architecture provides three primary benefits:

1. **Scalability:** Independent scaling of storage (ADLS) and compute (Databricks) allows for resource optimization based on demand.
2. **Resilience:** Advanced error management in ADF and the preservation of raw data in the Bronze zone allow for easy recovery from failures.
3. **Performance:** Parallel ingestion and PySpark's distributed processing significantly reduce the "Time-to-Insight."

Future Enhancements

While production-ready, the document identifies two key areas for future iteration:

- **Security:** Transitioning from Access Keys to **Azure Managed Identities** to achieve a "Zero-Trust" security posture.
- **Velocity:** Moving from batch processing to continuous streams using **Spark Structured Streaming** for near real-time analytical capabilities.